Adaptive Explainable Hybrid Intrusion Detection Using Security-Rule Evidence, and Contextual-Bandit Verdict Adaptation

Saud Shaikh
Department of Computer Engineering
SVKM's NMIMS, Navi Mumbai, India
saud.shaikh303@nmims.in

Riddhima Ghule
Department of Computer Engineering
SVKM's NMIMS, Navi Mumbai, India
riddhima.ghule346@nmims.in

Gargee Thakur
Department of Computer Engineering
SVKM's NMIMS, Navi Mumbai, India
gargee.thakur347@nmims.in

Pratiksha Patil
Department of Computer Engineering
SVKM's NMIMS, Navi Mumbai, India
pratiksha.patil@nmims.edu

***Abstract*—Network intrusion detection balances the breadth of attack detection and false-alert load, against the evidence that analysts can review. We present a supervised benchmark and a hybrid intrusion-detection design integrating five classifiers—Random Forest, XGBoost, LightGBM, CatBoost, and a multilayer perceptron (MLP)—on CICIDS2017, CICIDS2018, and UNSW-NB15. In the provided results, LightGBM is adopted for CICIDS2017 (99.90% accuracy; 99.67% attack F1) and CICIDS2018 (98.0891% accuracy; 94.9974% attack F1), with Random Forest selected for UNSW-NB15 (81.4068% accuracy; 77.7574% attack F1). The UNSW-NB15 benchmark also demonstrates a useful operational trade: Random Forest can achieve 93.8541% attack recall with a 25.1867% false-positive rate. The earlier project manuscript shows a seven-rule evidence layer on the UNSW-NB15 test partition, with 15.9% attack recall and a 14.1% false-positive rate if any rule triggers. We submit those statistics as project results, though training artifacts and scripts for evaluation were missing from the provided archives and could not be reproduced. The adaptive decision is defined as a contextual bandit with Benign, Review, and Attack actions, not as an operational contextual bandit. The provided files do not attest a bandit experiment, an associated anomaly detector, SHAP, or dashboard; these are thus treated as unvalidated implementation artifacts. We distinguish the benchmark evidence from the integration proposal, and list the experiments needed to determine whether evidence fusion and feedback-driven verdict adaptation would enhance operational performance.**

***Index Terms*—intrusion detection, hybrid IDS, supervised learning, security rules, contextual bandits, explainable artificial intelligence, CICIDS2017, CICIDS2018, UNSW-NB15**

I. Introduction

Legacy networks for enterprise services, cloud services, IoT environments, and cyber-physical systems encounter diverse traffic and emerging attack techniques. An intrusion detection system (IDS) needs to recognize known attack signatures and detect new attack techniques that are not captured in existing signatures or training data. Signature-based detection can encode explicit information in rules and include human-interpretable explanations for alarms, but coverage depends on the types of signatures and the network features that are monitored. Machine learning (ML) classifiers based on supervised learning learn statistical classifiers trained on labelled data, but the classifiers' efficacy depends on the relevant training data set, distribution of classes, design of features, and closeness between data seen in evaluation and deployment. Anomaly detection identifies anomalies based on instances of outliers that are unlike existing examples, but outliers can be benign activity and lead to false positives. [1]–[5].

Thus the hybrid IDS can support the output of rules, anomaly scores, and supervised models as corroborating evidence rather than requiring it to be the only detection method. Related work on hybrid deep learning, anomaly detection via autoencoders, staged ML systems, and late fusion of machine-learning classifiers with Snort signature meta-data [2], [6]–[9] shows the development of meta-stages or hyperstages to combine the knowledge they provide. Given the related work, the research gap should not claim that hybrid IDSs have not yet been proposed, but that this project investigates a specific configuration that lets the hand-coded rule flags be traceable features and evaluates them along with supervised models with a feedback-tuning policy.

An analysts requires more than a once-off alert. Analysts need to be provided with evidence of conditions of flow or features in the model that have led to the detection. The CS layer reports hits of the actual rules and SHapley Additive exPlanations (SHAP) is a suggested approach for explaining supervised-model outputs [10], [11]. The two forms of explanation were complimentary: rule hits defined conditions explicitly; SHAP assigned an output to the presence of particular inputs. Explanations should not be taken as convincing evidence of correctness, the predictive model must be verified separately.

This report describes the present project, separating out outcomes of measured work from additional proposed integrations. A supervised baseline is run on 3 cleaned datasets.

The CS rule layer is run and assessed on the cleaned UNSW-NB15 test partition. The provided rule-layer report offers no experiments on rule-plus-ML, with an unsupervised anomaly branch, with SHAP explanations, or with an RL policy, and so those are marked as architectures and evaluation goals unless logs and results are entered there.

The contributions of this paper are:

- five-model supervised benchmark on CICIDS2017, CICIDS2018 and UNSW-NB15, with the previous results of evaluation saved in the project; CS layer including feature proxies, per rule metrics and a false positive analysis; An adaptive hybrid architecture, where evidence of rule and the output of the model are separated and contextual bandit is used for adaptation at the policy level.
- A reproducible evaluation plan to evaluate the rule layer, supervised model and the combined system, without ascribing unmeasured gains to our proposed fusion.

II. Related Work

A. Hybrid and Multi-Stage Intrusion Detection

Expert model/phase hybrid IDS research combines complementary models or stages to take advantage of differences in their strengths. Tang *et al.* combine LightGBM with an autoencoder motivated by combining supervised discrimination with learned representations [2]. Abdallah *et al* study a hybrid CNN–LSTM approach to anomaly detection in software-defined networks, illustrating temporal and spatial feature learning in network monitoring [1]. Altunay and Albayrak study a CNN–LSTM-based IDS for industrial internet of things networks, and Henry *et al* study hybrid deep learning and feature optimization [6], [12]. All of these examples are mostly model-level hybridization; the current project's CS layer itself offers a third source of hand-authored, human-readable rule evidence.

Study has been made of hybrid schemes for IoT and medical IoT contexts. Salehpour *et al* use XGBoost, a Random Forest using ADASYN data augmentation, a cloud-based framework for IoMT traffic [7]. Yang *et al* use a combination of LightGBM and MobileNetV2 in a two-stage IoT IDS [8]. Akif *et al* look at hybrid ML schemes for IoT intrusion detection [13]. These papers give some reasons to think an ensemble, or staged, approach makes sense here, but would not guarantee that a particular approach would be better than the existing one: an ablation study on the splits of this project could confirm this.

Wen-Udeoji *et al.* considers late-fusion stacking of ML outputs and Snort-derived signature metadata on UNSW-NB15 [9]. This is particularly relevant because the signature evidence is explicitly incorporated with learned evidence. The current project should therefore frame the contribution tightly: implement and evaluate the seven-rule CS layer of the current project, with its traceable per-flow rule hits, as well as a proposed adaptive policy that can incorporate rule- and model-based evidence. The novelty of this should be determined against the exact fusion procedure, feature representations, and experiments, not using the overarching concept of hybrid detection.

B. Anomaly Detection and Adaptive Modelling

Anomaly detection is concerned with automatically picking out any observation that was sufficiently different from the model of what is a typical or dominant behaviour. Autoencoders, Isolation Forest, clustering and semi-supervised methods have been proposed to contribute to intrusion detection given very limited available attack labelling [2], [3], [14]. An anomaly score can be combined with a supervised probability, but needs to be calibrated and tested: the normal increment of change in Service mix, network load or software behaviour may also look anomalous. A fusion system should quantify what the addition of the anomaly branch is actually adding.

Adaptation applies when the traffic distribution or the operational requirements evolve. Contextual-bandit techniques can choose between actions using features of the observed context and the observed feedback, not necessitating the long-horizon transition model of a full RL approach. For this project's purpose, the job is to choose a verdict of Benign, Review, or Attack depending on the evidence on the table and the analyst's response. The source code or feedback archives are not available to verify the exact context encoding, reward updates, exploration policy, and persistence behaviour; no adaptive-policy claim is made.

C. Explainable AI for Security

Lundberg and Lee as a general method to explain predictions of the model via attribution of features [10]. Hermosilla *et al.* compare the performance of SHAP and LIME in models with the goal of intrusion-detection, and highlight the role of explainability in forensic analysis [11]. For this project, rule hits already explain the CS branch. SHAP can provide feature-based explanations for a suitable supervised model; if used for a fusion policy, specify the method, and explain the function. Report examples of explanations only after the integration has been tried, and the output has been inspected.

D. Datasets and Evaluation Context

The project evaluates CICIDS2017, CICIDS2018 and UNSW-NB15. Variations in dataset-specific distributions and feature schemas mean comparing results across datasets is not straightforward. The UNSW-NB15 rule report also indicates that removing duplicates has greatly reduced the data partition relative to the original data set, also affecting class balance, preventing direct comparison with published results. Research on multiple datasets also shows how important it is to document the processing steps and evaluation splits exactly [15]–[19]. As a result, these results are not taken as leaderboard results in direct comparison, but as internal benchmarks against the project's own clean data partitions.

III. System Overview

A flow-processing trail is delineated in four different stages. These stages are feature normalization, explicit CS rules,

supervised prediction and adaptive decision handling. The concept of combining multiple hybrid detection stages was introduced previously, although the yield of this approach is highly model-, feature- and evaluation protocol dependent [2], [6], [9]. The plan is to use the anomaly detector in conjunction with a proposed explanation service. The separation is carefully thought out: an explanation may still be considered even when the model is wrong and the model probability is still calculable for an independent analysis.

A. Input and Schema Normalization

The CS service receives flow records and remaps dataset-specific columns to canonical labels. Canonical column names are lowercased alphanumeric strings that can then be looked up in an alias table. The dataset can have any number of packet-rate-related fields such as `rate` and `Flow Packets/s` mapping to a canonical rate, and any number of mean-packet-size-related fields such as `smean` or `Fwd Packet Length Mean` that map to a canonical mean packet size. CICIDS flow duration values are scaled from microseconds to seconds on the 10^{-6} scale. The interface also indicates whether each rule runs at full, reduced or disabled feature sets, exposing missing feature dependencies to the analyst.

B. Classical Signature Rule Layer

Seven rules are hand-coded in the CS layer: DDoS-like rate, port-scan proxy, small-packet flood, exploit proxy, backdoor proxy, shellcode proxy, worm-like high-rate multi-port traffic. Every flow has seven binary flags along with an aggregate flag:

$$\hat{Q}_{t+1}(c,a) = \hat{Q}_t(c,a) + \frac{1}{N_t(c,a)}\left[r_t - \hat{Q}_t(c,a)\right] \quad (1)$$

There is a `cs_rule_hits` field that contains the names of the rules that were triggered. The thresholds are stored in a configuration file, so they can be modified in a configuration file without editing the rule code. This enhances traceability and configurability, but does not ensure portability across datasets or live networks.

C. Supervised ML Layer

Five models were benchmarked: Random Forest, XGBoost, LightGBM, Cat Boost, and MLP, representing tree ensembles, gradient-boosting implementations, and a neural baseline [20]–[23]. This saved evaluation task is a binary classification: benign or attack. These models are trained on labelled examples, and their results are presented with accuracy, attack precision, attack recall, attack F1-score, false-positive rate (FPR), and ROC-AUC. The datasets are from different collection and design pipelines, so their data schemas and evaluation contexts are described separately [17]–[19]. In addition to accuracy, a model with high attack recall but high FPR may be too much for analysts to handle, so model choice should include more than accuracy alone.

D. Anomaly Detection, Explainability, and Analyst Interface

Anomaly evidence and SHAP explanations are part of the proposed design, and a Streamlit dashboard is mentioned in the project description. However, there was no source code, anomaly scores, SHAP images, or dashboard images or code found in the two provided archives, so their status of integration cannot be determined based on what was examined. No detection improvements are attributed to these parts in the manuscript.

If global explanations are produced, they should be reported to provide an overview of the contribution of features to the difference (among features) for a given evaluation sample. Local explanations should provide a description of the contribution of features to a particular flow. The explanations should use the same feature names, ordering and artifacts used in the model. A SHAP explanation is not a cause account, and does not by itself provide a causation for a feature causing an attack.

If there is an anomaly detector, the last paper must include it and the specifics of its algorithm, training/test split, fitting process, score-calibration, threshold-selection, and how it influences the final verdict. High anomaly scores are not evidence of malicious behavior. Other benign behavior is "anomalous" and could get a high value as well. Dashboard features such as upload CSV, pick a model, show verdict, create explanation, and send analyst feedback should be explained only after they have been implemented in the application source code or demonstrated in screenshots. Those elements are seen as the implementation-verification items for now.

IV. Contextual-Bandit-Based Adaptive Verdict Policy

A. Scope and status

The potential adaptive element is a contextual-bandit-type policy, which uses a small amount of evidence about a flow to choose among three analyst-facing verdicts: *Benign* (no further investigation needed), *Review*, or *Attack*. This is a weaker setting than a full reinforcement-learning formulation in general, but the described implementation uses only feedback to learn estimates of the usefulness of actions given evidence context, and does not learn a transition model or optimize for hitherto unexplored state-action values over an extended horizon. The relevant archives include the prior manuscript as well as the literature collection, but the application source code and feedback logs are not included. As a result, we describe the policy here as a project design document and do not substantiate its empirical effectiveness.

B. Context, actions, and feedback

A context will contain the evidence that is available for a flow, for example the prediction from a supervised model, the rule-hit scores, and the anomaly score, but only if an anomaly detector is hooked up. A conceptual context might be expressed as.

$$x_t = [\hat{y}_t^{\mathrm{ML}},\ \mathbf{r}_t,\ a_t] \quad (2)$$

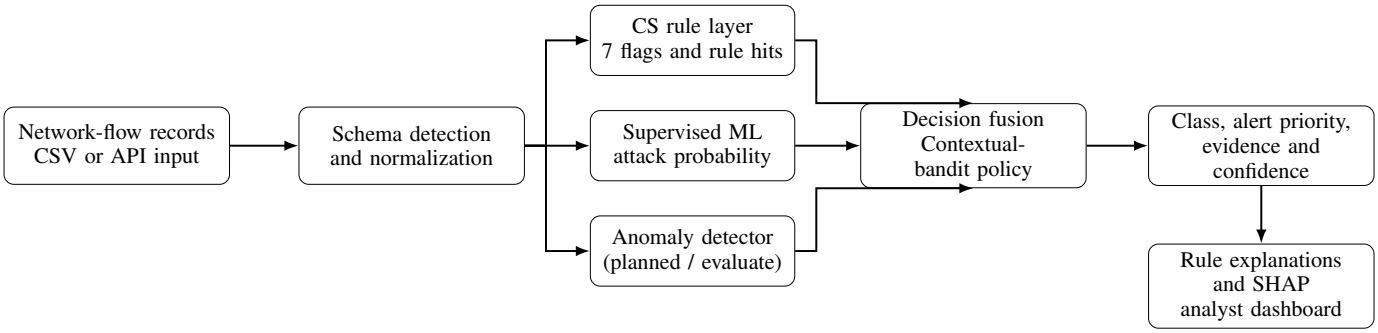


Fig. 1. Architecture of the adaptive explainable hybrid intrusion detection system. The cybersecurity rule layer and supervised models have evaluation results. Components labelled as planned or requiring validation should not be interpreted as experimentally verified.

Where $\hat{y}^{ML}t$ is the model output, $\mathbf{r}t$ is rule evidence available, and a_t is an available anomaly signal. This is simply an example and should be replaced with the proper encoding with the completed implementation viewed. The action set is

$$\mathcal{A} = \{\text{Benign}, \text{Review}, \text{Attack}\} \tag{3}$$

The project mentions two example rewarding results: Attack verdict accepted by the analyst is rewarded as $+1$, while Attack verdict rejected by the analyst is rewarded as -1. Rewards for Benign and Review, handling of missing feedback, and feedback-based labelling protocol are unspecified and cannot be simulated.

If the implementation stores a running mean reward for each context-action pair, a generic update would be

$$Q_{t+1}(x,a) = Q_t(x,a) + \frac{1}{N_t(x,a)}\left(r_t - Q_t(x,a)\right), \tag{4}$$

Where $Nt(x,a)$ is the number of times that pair was observed. We include this as it is a typical illustrative form for the update; we do not claim that it is the specific one used by the project. The source code will need to identify if estimates are as it or just $Tt(x)$, and whether the policy employs exploration, tie-breaking, persistence and cold-start rules.

C. Evaluation required

An appropriate adaptive-policy evaluation compares the policy to fixed verdict baselines on the same held-out feedback stream. It reports attack precision and recall, false-positive rate, review volume, reward under a predeclared reward definition, and sensitivity to delayed, noisy, or sparse analyst feedback. Training and evaluation feedback are separated to prevent data leakage. Without such an experiment, none of the accuracy, false-positive rate, analyst workload, or security improvements can be assigned to contextual-bandit adaptation.

V. Experimental Methodology

A. Data Partitions

The saved supervised benchmark consists of 1,089,279 CICIDS2017 test rows, 61 features; 39,981 CICIDS2018 test rows, 60 features; 48,351 UNSW-NB15 test rows, 183 encoded features. Both original dataset papers choose different traffic-generation settings and feature construction, highlighting the importance of recording the preprocessing and partition split when reporting results [17]–[19]. Evaluation of the CS rule is performed on the cleaned UNSW-NB15 test partition, with 48,351 flows of which 16,743 are attacks and 31,608 are benign. The CS report observes that removal of duplicate flows strongly affects the UNSW-NB15 partition, reducing the number of flows by 311, a third of the original class 0 flows. The class distribution is therefore different, and these metrics should not be directly compared with results on the other split without qualification.

B. Metrics

Let TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives, with attack treated as the positive class. Metrics are calculated as

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}, \tag{5}$$

$$Precision = \frac{TP}{TP + FP}, \qquad Recall = \frac{TP}{TP + FN}, \tag{6}$$

$$F1 = 2\frac{Precision \cdot Recall}{Precision + Recall}, \tag{7}$$

$$FPR = \frac{FP}{FP + TN}. \tag{8}$$

These measures provide measures of various angles of the performance and should be used in conjunction, instead of justaccuracy calculations. The CS layer report calculates the measures per rule and for union of all rules. Per class coverage was not calculated as the cleaned rule-evaluation files do not have the attack-category column and hence, it needs to be fixed before arriving at any individual attack categories covered/missed.

C. Evaluation Protocol

The current paper uses the kept model metrics and the CS-layer report provided with the deliverable. The combined rule-plus-ML experiment has not been provided, so the paper does not make any claim of improvement for this hybrid system. The next more rigorous experiment to compare (1) each supervised model without rule flags, (2) same model

with CS flags appended, (3) rule layer only, (4) anomaly detector only, (5) fixed OR/weighted-score fusion, and (6) learned or contextual-bandit-based policy, should use the same split and prep. Model-selection and threshold setting should be performed on training and validation data only; test set should be held out for final test.

D. Operational Meaning of Adaptation

For the purposes of the project, please operationalize adaptation as an observable decision for which you can test. Simply picking the best model once as a point in a sample table of a benchmark is dataset-specific model selection, not an online contextual bandit. A real RL component would require you to have an agent of some sort that observes a state, picks an action, gets a feedback, then updates or otherwise produces a new policy according to a learning algorithm [24], [25]. That matters for the claims in the paper: if our current code is just using a pre-calculated table to select LightGBM for one dataset and Random Forest for another, that is an adaptive selection rule but not, by itself, an contextual-bandit policy. If that contextual-bandit policy exists somewhere else in the project, report its state, action, reward, training episodes, and evaluation performance.

An IDS, however, faces its use of feedback in delay (alert evidence may not be attributed to a given alert until subjected to analyst review, false negatives may be only identified post-incident, etc.). Reward construction thus becomes one of the design choices. The convenience of a simulated reward derived directly from test labels can help in offline development of algorithms, but will not be presented as feedback available in a deployed system. An effective evaluation methodology for deployment must clearly distinguish the test labels used in the training set from those for final evaluation and elucidate the mechanism of a deployed agent receiving feedback. Until these aspects are documented, the RL contribution remains as a form of adaptive control design goal.

E. Relationship Between the Rule Layer and Policy Layer

An RL policy and CS rule layer serves a different purpose. CS rules are rules, a condition when a flow's rate exceeds the configured value and connection state matches a particular set and the return value is a flag "triggered DDoS like attack flag". This rule doesn't learn from a reward, nor is it rebuilt once the input and rule configuration are the same. The RL policy also maps an observation to an action, and is therefore free to change the action based on the feedback; the pattern matching part of the observation could include the various CS flags, but it should not benignly override the rule definitions, unless this action has been explicitly implemented.

Separating the output is also beneficial for auditability. Each alert should have the raw rule hits, supervised model score, anomaly score if relevant, and policy action recorded. Policy recommendations should be clearly differentiated from detected facts: the fact that 'Rule 1 fired' is a traceable deterministic outcome while 'the policy escalated this flow because the current operating state favours review' is a decision made by the adaptive controller. Both should be recorded so that an analyst can reconstruct how the alert was generated, and offline error analysis can be supported.

F. Model Calibration and Threshold Selection

The supervised output is a probability-like value. However, interpreting a model score as a calibrated probability comes with caveats. Calibration on validation data can be assessed using reliability diagrams, Brier score or expected calibration error. Thresholding should align with the cost of false negatives versus false positives. Using a low threshold on supervised models can improve attack recall at the expense of volume, while a high threshold can impose a safeguard against false alerts at the expense of attack recall. This is important for UNSW-NB15 in which the saved results show FPR values greater than 25% in all five supervised models.

An adaptive policy may eventually use traffic context and recent validated feedback to recommend threshold changes. However, any such changes should be bounded and evaluated on a validation stream that is separate from the final test data. The paper should report the baseline threshold, the selection procedure, and the exact operating point used for each table. If different models use different thresholds, this must be documented because it affects comparability. A model should not be called "best" without stating the criterion; this paper uses accuracy and attack F1-score to describe the strongest baselines on CICIDS2017 and CICIDS2018, while explicitly noting other trade-offs on UNSW-NB15.

VI. Results

A. Supervised Model Benchmark

Tables I–III Summarize the saved evaluation results for the project. Accuracy, precision, attack recall, attack f1, and FPR are all percentage values, the ROC-AUC has no unit. Dashes show values not provided in project summary. The RF FPR for the complete set of CICIDS2017 figures has not been provided in the sample project summary file, as its value does not appear in the results CSV, it must be filled from that file rather than be guessed.

The model with the best accuracy and attack F1-score in the listed results for CICIDS2017 is LightGBM. The values of the attack recall reported for LightGBM is 99.57% and the FPR is 0.04% which is the only value of FPR reported in the benchmark summary list for all of the tested models, therefore the FPR is only shown for LightGBM. The high values of the ROC-AUC scores for the models suggest the models ranked well for this particular cleaned test partition, but this does not demonstrate they did well on live traffic or data out of sample. This concern is consistent with the discussion on closed-world assumptions and dataset limitations of ML-based intrusion detection. [4], [17].

On CICIDS2018, LightGBM achieves 98.0891% accuracy and 94.9974% attack F1-score. Attack recall is approximately 90.68%, meaning some malicious examples remain undetected. The result highlights why recall and FPR should be reported alongside accuracy.

TABLE I
SUPERVISED RESULTS ON CICIDS2017

Model	Accuracy	Precision	Attack Recall	Attack F1	FPR	ROC-AUC
Random Forest	99.85	99.70	99.28	99.49	–	0.9998
XGBoost	99.86	99.77	99.29	99.53	–	1.0000
LightGBM	99.90	99.76	99.57	99.67	0.04	1.0000
CatBoost	99.80	99.78	98.92	99.35	–	0.9998
MLP	99.30	98.43	96.94	97.67	–	0.9990

TABLE II
SUPERVISED RESULTS ON CICIDS2018

Model	Accuracy	Precision	Attack Recall	Attack F1	FPR	ROC-AUC
Random Forest	97.6714	97.3603	90.8250	93.9792	–	0.976219
XGBoost	98.0341	99.6149	90.5250	94.8527	–	0.986299
LightGBM	98.0891	99.7525	90.6750	94.9974	0.0563	0.986633
CatBoost	98.0041	99.5460	90.4375	94.7734	–	0.984168
MLP	97.8565	98.8110	90.3750	94.4049	–	0.983499

TABLE III
SUPERVISED RESULTS ON UNSW-NB15

Model	Accuracy	Precision	Attack Recall	Attack F1	FPR	ROC-AUC
Random Forest	81.4068	66.3738	93.8541	77.7574	25.1867	0.950979
XGBoost	81.0345	65.6603	94.8217	77.5915	26.2687	0.957316
LightGBM	81.1855	65.8894	94.6843	77.7051	25.9649	0.957890
CatBoost	78.8629	62.7238	96.0282	75.8826	30.2297	0.953703
MLP	77.3179	61.0796	95.0905	74.3816	32.0963	0.935120

UNSW-NB15 exhibits an even more distinct operating profile. Random Forest achieves the best accuracy and attack F1-score among these reported metrics, but XGBoost achieves the highest attack recall, and LightGBM the best ROC-AUC. The false positive rate also remains around 25–32% across models. This is a primary motivation for calibrating thresholds, class-wise errors, and evidence from complementary rules rather than assuming one classifier is always the optimal.

B. Classical Signature Rule Results

Table IV reports the implemented CS rule layer on the cleaned UNSW-NB15 test partition. The union of the seven rules flags 7,113 flows, of which 2,666 are true positives and 4,447 are false positives. The resulting precision is 37.5%, recall 15.9%, F1-score 22.4%, and FPR 14.1%.

The four rules exhibit very high precision but very low recall. No rule by itself has more than a single-rule recall of 15.1%. Rule 6 proved weak in this test because the sample has many small non-HTTP flows among benign data. The union's precision was only marginally higher than the dataset's 34.6% attack prevalence, so the rules should not be offered as a standalone detector. Their potential advantages may be in transparency of evidence and a low false alarm rate, and as a feature-level input to a fusion model. Whether this improves ML remains an empirical question that could be answered by an ablation experiment.

VII. DISCUSSION

The supervised results vary across datasets. LightGBM is strongest by accuracy and attack F1-score on CICIDS2017 and CICIDS2018, but UNSW-NB15 has high false-positive rates for all five classifiers. The rule layer also has limitations: its overall recall is low, and several conditions use proxies because the cleaned UNSW-NB15 features do not include raw TCP flags or destination-port fields. For example, the port-scan rule uses a connection-count feature divided by duration; this may inflate the score for very short flows. Likewise, treating non-HTTP service as a shellcode proxy creates false alerts on legitimate DNS and other non-HTTP traffic.

These findings imply a more modest role for each component. Supervised ML can offer wide discrimination derived from training data, rules can offer direct but partial inference, anomaly detection could indicate divergence from the learned norm, and an adaptive policy can decide what to do with those inputs. The components should not be combined until their outputs can be judged on the same examples, and their respective scales and calibration can be established. A learned meta-model should be trained with out-of-fold base-model predictions, or on a validation set, to avoid leakage from in-sample predictions.

It is fair to state at this point that the full hybrid system does not demonstrate an improvement over the supervised models. An earlier hybrid and signature-fusion work is a motivation to test complementary evidence, but it is not an ablation of the current implementation and the data splits used [2],

TABLE IV
IMPLEMENTED CS RULE PERFORMANCE ON UNSW-NB15

Rule	Flagged	TP	FP	Precision	Recall	F1	FPR
R1 DDoS-like rate	3,938	2,525	1,413	64.1%	15.1%	24.4%	4.5%
R2 Port-scan proxy	3,833	1,756	2,077	45.8%	10.5%	17.1%	6.6%
R3 Small-packet flood	2,837	1,117	1,720	39.4%	6.7%	11.4%	5.4%
R4 Exploit proxy	36	30	6	83.3%	0.2%	0.4%	0.02%
R5 Backdoor proxy	411	122	289	29.7%	0.7%	1.4%	0.9%
R6 Shellcode proxy	498	82	416	16.5%	0.5%	0.9%	1.3%
R7 Worm-like traffic	1,259	1,037	222	82.4%	6.2%	11.5%	0.7%
Any rule	7,113	2,666	4,447	37.5%	15.9%	22.4%	14.1%

[9]. The available CS report specifies that the rule-plus-ML hybrid has yet to be performed. Likewise, no anomaly detector result, SHAP example, or RL policy comparison was provided; those are rather important missing artifacts, not just cosmetic omissions: they make it possible to say whether the proposed integration is profitable.

VIII. REPRODUCIBILITY AND IMPLEMENTATION DETAILS

The CS layer is implemented as a rule module, an evaluation module, a FastAPI service, a browser interface, and tests. The rule module provides an applyallrules(df) function which identifies the existing columns, applies the configured conditions, and adds the rule flags, aggregate flag, and rule-hit string. The evaluation module computes per-rule true positives, false positives, false negatives, precision, recall, F1-score, FPR, and per-class coverage when the attack-category field is present. The API exposes analysis endpoints for uploaded flows, a demo, retrieval or update of configuration, and JSON scoring. Unit tests demonstrate that the implemented rules trigger for constructed flows and that the results for missing columns do not produce anomalous alerts. These implementation details should be sufficient to replicate the CS rule layer, given that the code, configuration, dependencies, and test input files are preserved for the final submission.

The rule thresholds are: Packet-rate threshold of 1,000 for the DDoS-like rule, Port-scan proxy threshold is 1,000/sec with min duration of 0.001 sec, Small-packet flood is mean packet-size below 60 bytes with rate above 500 packets/sec. Exploit proxy uses 1,400-byte mean packet-size threshold or URG flag if present, Backdoor proxy uses duration of $\geq$ 10 sec, rate of $<$ 5 per sec, and unrecognized service, Shellcode proxy flags total bytes below 100 on non-HTTP service, Worm proxy uses rate above 1,000/sec, mean packet-size below 60 bytes, at least 2 destination ports.. These thresholds are justified with the project overlap analysis, and must be justified as heuristics, not as evidence of the exact signature for that attack family.

One major limitation in implementation, caused by the underlying features, is that the UNSW-NB15 cleaned features set contains no raw TCP flags and no destination-port numbers. Some of the rules have proxy conditions: an incomplete handshake cannot be checked directly, so a connection state monitors incomplete handshakes; a connection-count divided by duration monitors scan intensity; and a service name monitors a port class. While monitoring a port class is useful for proof of concept, it may not carry the semantics of the original rule condition. The implementation should therefore report rule mode (full, reduced, disabled) per dataset and should not compare rule performance between datasets without describing the differences in inputs available.

To complete the experimental record, the final artifact should include: dataset source and version, hash or other stable identifier for the cleaned files, code for pre-processing, list of feature-columns, label mappings, list of train/val/test split procedure, list of duplicate-removal procedure, list of hyperparameter for models, versions for libraries, seeds, and running environment info. For RL, it should also include: algorithm name, reward equation and coefficients, constraints on actions, construction of episodes, training curves, baseline policy, and evaluation on held-out data. For SHAP, it should also include: model explained, type of explainer, background dataset or reference distribution used in the explainer, and examples used for paper. Such level of reporting makes transparent where the results are from the base classifiers versus rule or adaptive component.

IX. LIMITATIONS AND THREATS TO VALIDITY

First, the characteristics of the dataset cleaning and splitting strategy will affect measured performance: survey of datasets and evaluation procedures shows that dataset construction and evaluation procedures can significantly impact the relative comparison of different IDS studies [4], [17]. The UNSW-NB15 cleaned data split has been heavily duplicate-removed so class balance differs from the original dataset; second, the CS rule conditions include approximations as some fields are unavailable; third, binary supervised labels do not allow claims of the attackcategory granularity or zero-day detection. Results on offline flow records do not speak to real-time throughput, latency, or robustness to adversarial manipulation; fourth, the features and task formulation prevent conclusions of the contextual-bandit policy's algorithm, state, action, reward, or evaluation, leaving the adaptive benefit unmeasured; finally, the explainability methods simply describe model behaviour but do not provide verifiable explanations of the factors in learned behaviour, so does not address causal reasoning, and SHAP provides a feature-attribution explanations that is not equivalent to causal effect explanations. [10], [11].

X. Future Evaluation Plan

The following experiments should follow. 1. Incorporate the CS flags into the same supervised training and test system, compare the same classifiers with and without the eight rule-derived features. 2. Evaluate each rule on its own and together on the real files from CICIDS2017 and CICIDS2018 using real dataset files with full, reduced, or disabled rule modes. 3. Implement and validate a single anomaly detector with a explicit training split and a explicit anomaly threshold. 4. Specify the exact RL policy algorithm and reward function, train without seen-of-held-out labels at decision time, compare to fixed-threshold and fixed-fusion baselines. 5. Add the SHAP explanation with representative correct and incorrect example, then measure inference latency and throughput before calling it real-time.

The core comparison experiment should compare rule only, anomaly only, supervised only, fixed fusion, adaptive fusion all under the same held-out protocol. The report should include: precision, recall, F1-score, FPR, ROC-AUC where relevant, confusion matrix, calibration, per-attack results if labels available. For RL: learning curve, final evaluation reward, distribution of chosen actions, variance across random seeds, safety analysis. Such an evaluation would confirm or deny that adaptation leads to a lower miss rate / false alert ratio.

XI. Conclusion

This paper presents a supervised-learning baseline and a re-implemented Classical signature-rule layer for an Adaptive Explainable Hybrid Intrusion Detection System (IDS). The research compares five machine-learning Models The CNN models across CICIDS2017, CICIDS2018 and UNSW-NB15.. Importance of dataset-aware evaluation in intrusion detection [4], [5], [17]. LightGBM achieves the strongest

Different accuracy and attack F1-score for the models evaluated on CICIDS2017 And CICIDS2018. Nevertheless, these results on UNSW-NB15 indicate a sizable. False-positive trade-off, demonstrating that model performance depends On the nature of the dataset and the traffic being modeled [17].

The seven-rule cybersecurity layer provides transparent and Reproduce rule-based evidence. The recall is decent, but not enough. and the false-positive rate is 14.1%. So, the rule layer is better suited to complementing supervised machine-learning predictions Greater than to being an independent intrusion detector. Hybrid detection May fuse complementary sources of evidence, or the.

Future work will incorporate supervision with rule evidence.

predictions, evaluating anomaly-detection scores, incorporating

explainability through SHAP [10], and validating a Contextual-bandit policy that adapts verdicts based on analyst feedback Selection. The whole hybrid architecture should be tested using. A consistent, leakage-safe experimental protocol, reporting precision, Recall, F1-score, false-positive rate, ROC-AUC where available, and Confusion matrices. These experiments are essential to ascertain I.e., the applied set of tools offers tangible benefits compared to. Its individual components.

Acknowledgment

The authors express their sincere gratitude to Prof. Pratiksha Patil for her valuable guidance and support throughout this research. The authors also acknowledge her Research Methodology course, which provided the academic framework within which this research was conceived and developed.

References

[1] M. Abdallah, N. A. L. Khac, H. Jahromi, and A. D. Jurcut, "A hybrid CNN-LSTM based approach for anomaly detection systems in SDNs," in *Proceedings of the 16th International Conference on Availability, Reliability and Security (ARES)*, 2021.

[2] C. Tang, N. Luktarhan, and Y. Zhao, "An efficient intrusion detection method based on LightGBM and autoencoder," *Symmetry*, vol. 12, no. 9, p. 1458, 2020.

[3] R. Almuhanna and S. Dardouri, "A deep learning/machine learning approach for anomaly-based network intrusion detection," *Frontiers in Artificial Intelligence*, vol. 8, 2025.

[4] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," *2010 IEEE Symposium on Security and Privacy*, pp. 305–316, 2010.

[5] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys and Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.

[6] H. C. Altunay and Z. Albayrak, "A hybrid CNN+LSTM-based intrusion detection system for industrial IoT networks," *Engineering Science and Technology, an International Journal*, vol. 38, p. 101322, 2023.

[7] A. Salehpour, M. Norouzi, M. A. Balafar, and K. SamadZamini, "A cloud-based hybrid intrusion detection framework using XGBoost and ADASYN-augmented random forest for IoMT," *IET Communications*, 2024.

[8] Y.-M. Yang, K.-C. Chang, and J.-N. Luo, "Hybrid neural network-based intrusion detection system: Leveraging LightGBM and MobileNetV2 for IoT security," *Symmetry*, vol. 17, no. 3, p. 314, 2025.

[9] S. C. Wen-Udeoji, M. K. Muyeba, and A. Mohammadi, "Late-fusion stacking of machine learning and snort signatures for multi-class intrusion detection on UNSW-NB15," *Artificial Intelligence and Applications*, 2026.

[10] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[11] P. Hermosilla, S. Berríos, and H. Allende-Cid, "Explainable AI for forensic analysis: A comparative study of SHAP and LIME in intrusion detection models," *Applied Sciences*, vol. 15, no. 13, p. 7329, 2025.

[12] A. Henry, S. Gautam, S. Khanna, K. Rabie, T. Shongwe, P. Bhattacharya, B. Sharma, and S. Chowdhury, "Composition of hybrid deep learning model and feature optimization for intrusion detection system," *Sensors*, vol. 23, no. 8, p. 890, 2023.

[13] M. A. Akif, I. Butun, A. Williams, and I. Mahgoub, "Hybrid machine learning models for intrusion detection in IoT: Leveraging a real-world IoT dataset," 2025.

[14] A. Haque and H. Soliman, "A transformer-based autoencoder with isolation forest and XGBoost for malfunction and intrusion detection in wireless sensor networks for forest fire prediction," *Future Internet*, vol. 17, no. 4, p. 164, 2025.

[15] H. Xuan and M. Manohar, "Intrusion detection system with machine learning and multiple datasets," 2023.

[16] M. Arcos-Argudo, R. Bojorque, and A. Torres, "A deterministic comparison of classical machine learning and hybrid deep representation models for intrusion detection on NSL-KDD and CICIDS2017," *Algorithms*, vol. 18, no. 7, p. 749, 2025.

[17] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers and Security*, vol. 86, pp. 147–167, 2019.

[18] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "A detailed analysis of the CICIDS2017 data set," in *Information Systems Security and Privacy*, ser. Communications in Computer and Information Science, P. Mori, S. Furnell, and O. Camp, Eds. Cham: Springer, 2019, vol. 977, pp. 172–188.

[19] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *2015 Military Communications and Information Systems Conference*, 2015, pp. 1–6.

[20] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[21] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[22] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[23] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[24] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[25] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proceedings of the 19th International Conference on World Wide Web*, 2010, pp. 661–670.